

CSE234 Week 4: STL List and Deque

This week, we are discussing the STL List and Deque classes. These are the remaining sequential container classes provided by the STL.

List

The STL List class is used to create a doubly linked list. Similar to a vector, lists store a collection of elements while preserving their order. Linked lists are different from vectors in that the elements can no longer be accessed by an index and are not stored contiguously in memory. This can be an advantage or disadvantage depending on the usage. Rather than using an array, which is contiguous in memory, arrays use nodes which store pointers to the previous and next elements. Full implementation details of a list are a topic for a data structures class.

Similarities to Vector

Lists support many of the same operations as vectors, minus the subscript operator `[]`. Lists add the `push_front()` and `pop_front()`, in addition to the `push_back()` and `pop_back()` member functions. This allows direct access to the front of the list, whereas vectors only allow access to the back. They support all the other member functions of vector except for `capacity()`.

Unique Operations

Lists provide a few unique member functions for common list operations. First is `splice()`, which is used to combine two lists (or sub-lists) into one. Splicing allows elements of lists to be transferred between each other, or reordered within the same list, without the overhead of copying.

The `merge()` member functions assumes that you have two sorted lists, and merges them into a single sorted list.

The `remove()` function searches for the value that is passed to it, and

removes it from the list. This is different from `erase()`, which expects an iterator to the element to be removed. The `remove_if()` function takes a function pointer/functor/lambda and removes items that satisfy the condition of that function. Don't worry too much about this for right now; function pointers/functors/lambda's are a point of discussion for a later week.

Lists also offer a `reverse()` member function, that reverses the contents of the list, and `sort()` which sorts the list.

The `unique()` function removes consecutive duplicated elements. For example, `std::list<int>({1, 2, 3, 3, 4, 5, 5, 2, 3}).unique()` would result in the list `{1, 2, 3, 4, 5, 2, 3}`. The consecutive 3s and 5s are removed, but the extra 2 and 3 at the end of the list stays.

Common Applications

Lists are used when there are frequent needs for insertions or removals, or to move elements between lists via `splice()`.

Example

A cache is used to store recently accessed items. Many caches use LRU (least recently used) to decide which item to evict from the cache when it gets full. One example of a cache is the CPU cache. Your computer's CPU has a small amount of memory that is faster than RAM and is part of the CPU. The most recently accessed data is stored in this cache, as it is assumed that recently used data will be accessed again soon. Another example of a cache is a browser cache. Recently accessed webpages, images, etc. are stored locally on your computer. This way, they will be available faster when you access them again.

This example shows a class that implements an LRU cache using a list as the internal cache representation. For the example we'll just use integers for data. While the cache has a fixed size, the `splice()` member makes moving elements within the cache faster than it would be with a vector or array.

```
#include <iostream>
#include <list>

// class that implements LRU Cache using an std::list
class LRUCache {
```

```

public:
    LRUCache() {
        maxCacheSize = 64;
    }
    LRUCache(size_t size) {
        maxCacheSize = size;
    }
    // checks for a cache hit, i.e.,
    // if the data is in the cache
    bool hit(int data);
    // returns an iterator the data in the cache
    // if the data is not in the cache, it is added
    // if the data is in the cache, it is moved to
    // the front of the cache
    // if the cache is full, evicts LRU element
    // to make room for new item
    std::list<int>::iterator get(int data);
    // get the size of the cache
    size_t size() const { return maxCacheSize; }
private:
    // helper function to find an item in the list
    std::list<int>::iterator find(int data);
    // list for the cache
    std::list<int> cache;
    // limit to size of cache
    // if a cache is too big, it will become slower to
    // search and won't be as useful
    size_t maxCacheSize;
};

std::list<int>::iterator LRUCache::find(int data) {
    auto it = cache.begin();
    while (it != cache.end()) {
        if (*it == data) {
            return it;
        }
        ++it;
    }
    return it;
}

bool LRUCache::hit(int data) {
    return find(data) != cache.end();
}

std::list<int>::iterator LRUCache::get(int data) {
    // get iterator to data item
    auto it = find(data);
    // if it is found
    if (it != cache.end()) {
        // move to the front of the cache using splice

```

```

        cache.splice(cache.begin(), cache, it);
        // and return the iterator
        return it;
    }

    // if not, check if the cache is not full
    if (cache.size() < maxCacheSize) {
        // if it is not full, just push the new data to the front
        cache.push_front(data);
    }
    else {
        // remove the last item of the cache
        cache.pop_back();
        // add the data to the front
        cache.push_front(data);
    }

    // in both cases, we return the first iterator of the list
    return cache.begin();
}

int main() {
    LRUCache cache(4);

    cache.get(1);
    cache.get(2);
    cache.get(3);
    cache.get(4);
    cache.get(1);
    cache.get(5);

    if (cache.hit(1)) {
        std::cout << 1 << " - Cache hit\n";
    }
    else {
        std::cout << 1 << " - Cache miss\n";
    }
    if (cache.hit(2)) {
        std::cout << 2 << " - Cache hit\n";
    }
    else {
        std::cout << 2 << " - Cache miss\n";
    }
    if (cache.hit(3)) {
        std::cout << 3 << " - Cache hit\n";
    }
    else {
        std::cout << 3 << " - Cache miss\n";
    }
    if (cache.hit(4)) {
        std::cout << 4 << " - Cache hit\n";
    }

```

```

    }
    else {
        std::cout << 4 << " - Cache miss\n";
    }
    if (cache.hit(5)) {
        std::cout << 5 << " - Cache hit\n";
    }
    else {
        std::cout << 5 << " - Cache miss\n";
    }
    return 0;
}

```

This will print:

```

1 - Cache hit
2 - Cache miss
3 - Cache hit
4 - Cache hit
5 - Cache hit

```

Stack and Queue

Before discussing the STL Deque, we'll briefly discuss stacks and queues. A stack is a container that only supports access to its elements in last-in, first-out (LIFO) order, like a stack of plates. A queue is a container that only supports access to its elements in first-in, first-out (FIFO) order, like a waiting queue at a cash register. The applications and implementations for stacks and queues are topics for a data structures course. The STL does provide stack and queue classes, but they are implemented as "container adapters". They simply limit the ways a vector, list, or deque can be accessed. The default STL stack and queue use deque, which is discussed next.

Deque

A deque is a double-ended queue. It is pronounced "deck", like a deck of cards, so as not to confuse it with "dequeue", which is typically the name of the operation that removes an element from a normal queue. A deque supports the same member functions as list, with the addition of the subscript operator and `at()` member function for random access to elements. Below is a table comparing vector, list, and deque:

	Vector	List	Deque
Random Access?	Yes	No	Yes
Push/Pop Front?	No	Yes	Yes
Push/Pop Back?	Yes	Yes	Yes
Implementation	Dynamic Array	Doubly Linked List	Doubly Linked List of Small Arrays
Cost of Insertion at Front	Copy entire array to shift elements	Insert single node at front	Add new node with subarray at front
Cost of Insertion in Middle	Copy part of array to shift elements	Insert single node at insertion location	Fill subarray element or insert new node with subarray
Cost of Insertion at End	Change element value unless the vector needs resized	Insert single node at end	Add new node with subarray at end
Cost to access random element	Fastest	Requires searching for the element	Requires list and array to be dereferenced, so slower than vector but faster than list

	Vector	List	Deque
Size	Wasted space due to automatic resizing, but on average less than deque	No wasted nodes, but each element contains the actual data and pointers to the previous and next element.	Most space usage, since it is a combination of a list (nodes with extra pointers) and arrays (which may have empty elements)

Deques are used when you need to add or remove elements to the front and back of the container frequently. Histories (eg. undo/redo and browser histories) can be efficiently implemented using deque.

Conclusion

Lists are used when frequent insertions and/or removals need to be made to the middle of a container. Lists use nodes, which contain pointers to the previous and next node, along with a data item, rather than an array. This means that the subscript operator cannot be used for random access to elements of a list. But, combining lists and reordering elements within a list is faster than with a vector. A deque is a double-ended queue, which uses a list of small arrays as its internal representation. This supports fast access to the front and end of the container, at the cost of using more space than a vector or list. Performance of a deque is typically between that of a list and vector for most operations.